ARSITEKTUR & RISET SISTEM AI CV SCREENER

Cetak Biru Rekayasa Perangkat Lunak: Implementasi Nyata & Rencana Pengembangan Masa Depan

Penyusun: Alif Ihsan
Afiliasi: AI7 Solaris Ecosystem
Klasifikasi: Cetak Biru Rekayasa Perangkat Lunak
Repositori: `agentic-cv-screener`
Status Komponen: **Eksisting (Aktif)** & **Rencana (Jev / Scaled DB)**
Versi: `v2.0-Production-Agentic`

DAFTAR ISI & STRUKTUR LAPORAN

1. ABSTRAK & RINGKASAN EKSEKUTIF 1
    1.1 Latar Belakang & Pernyataan Masalah 1
    1.2 Kegagalan Struktural Model Bahasa Generatif Konvensional 2
    1.3 Paradigma Solusi: Dekopel Persepsi Mikro & Agregasi 2
2. ARSITEKTUR SISTEM EKSISTING (TERIMPLEMENTASI) 2
    2.1 Diagram Topologi Arsitektur Sistem 3
    2.2 Mesin Pencarian Hibrida FX-32 (Dense HNSW + Sparse FTS) 3
    2.3 Deterministic Grounding Verifier & Zero-Hallucination Guard 4
    2.4 Autonomous ReAct AI Reasoning Engine & Telemetri State Machine 5
    2.5 Tata Kelola Keamanan, Sanitasi Dokumen, & Anti-Bias Compliance 6
    2.6 Antarmuka Manusia Sentuhan Presisi (*Tactile Anti-Slop Human Interface*) 7
3. ROADMAP & RENCANA MASA DEPAN (FUTURE WORK) 7
    3.1 Integrasi Calibrated Decision Models: Jev System One (TypeSafe 7
    3.2 Evolusi Basis Data Vektor: Analisis Arsitektural pgvector 9
    3.3 Multi-Modal Document Inspection & Otentikasi Forensik Visual 10
4. KESIMPULAN & IMPLIKASI DESAIN REKAYASA PERANGKAT LUNAK 10
    4.1 Sintesis Penemuan Riset 10
    4.2 Formulasi Matematis Inti Sistem 10
5. REFERENSI KAJIAN PUSTAKA 11

1. ABSTRAK & RINGKASAN EKSEKUTIF

1.1 Latar Belakang & Pernyataan Masalah

Proses seleksi awal berkas lamaran kerja (*Curriculum Vitae / Resume Screening*) dalam tata kelola rekrutmen berskala menengah hingga besar menghadapi ketidakseimbangan struktural antara volume pelamar yang eksponensial dan kapasitas kognitif manusia (*recruiter cognitive bandwidth*) yang terbatas. Dalam satu siklus rekrutmen posisi teknis spesifik, departemen *Human Resources* (HR) menerima ratusan hingga ribuan berkas lamaran, di mana waktu penelaahan awal per dokumen acap kali tereduksi menjadi hanya 6 hingga 10 detik. Reduksi waktu evaluasi manusia ini memicu bias heuristik seleksi, kelelahan kognitif, serta tingginya tingkat *false negative* terhadap kandidat kompeten.

Upaya otomatisasi gelombang pertama yang mengandalkan pencocokan kata kunci leksikal (*keyword matching* atau ekspresi reguler) terbukti sangat rapuh (*brittle*). Sistem tersebut menderita kebutaan semantik (*context blindness*), di mana kehadiran istilah teknologi pada bagian minat akademis atau hobi diperlakukan setara dengan pengalaman profesional bertahun-tahun. Sebaliknya, adopsi naif model bahasa besar generatif (*Generative Large Language Models / LLMs*) pada gelombang kedua memperkenalkan risiko sistemik baru: halusinasi bukti, distorsi atensi spasial, kerentanan injeksi perintah (*prompt injection*), pelanggaran regulasi privasi data pribadi, serta ketidakmampuan melakukan kalkulasi aritmatika skoring yang stabil dan deterministik.

1.2 Kegagalan Struktural Model Bahasa Generatif Konvensional

Penyerahan seluruh mandat penapisan dan pemeringkatan kandidat ke model generatif melalui instruksi langsung (*direct listwise prompting*) mengalami empat patologi matematis dan epistemologis fundamental:

1. **Halusinasi Kutipan & Asosiasi Fiktif (Citation Fabrication):** Model generatif beroperasi dengan memprediksi token berikutnya berdasarkan distribusi probabilitas teks:

   $$P(w_t \mid w_1, w_2, ..., w_{t-1})$$

   Model tidak memiliki mekanisme internal untuk menjamin bahwa sintesis teks yang diproduksinya merupakan representasi verbatim dari korpus sumber. Dalam pengujian empiris, LLM generatif kerap memparafrasekan atau bahkan mengarang kualifikasi fiktif kandidat demi memenuhi ekspektasi deskripsi pekerjaan (*hallucinatory extrapolation*).
2. **Degradasi Atensi Spasial ("Lost in the Middle" Effect):** Sebagaimana dirumuskan oleh Liu et al. (2023), mekanisme *multi-head attention* pada transformer mengalami penurunan bobot atensi signifikan pada interval tengah jendela konteks panjang ($x_{0.2L} ... x_{0.8L}$). Ketika puluhan CV dimasukkan ke dalam satu prompt gabungan, posisi kemunculan dokumen mendikte peluang kandidat untuk direkomendasikan (*primacy and recency bias*). Hal ini membatalkan aksioma keadilan perankingan permutasi invariansi (*permutation invariance*):

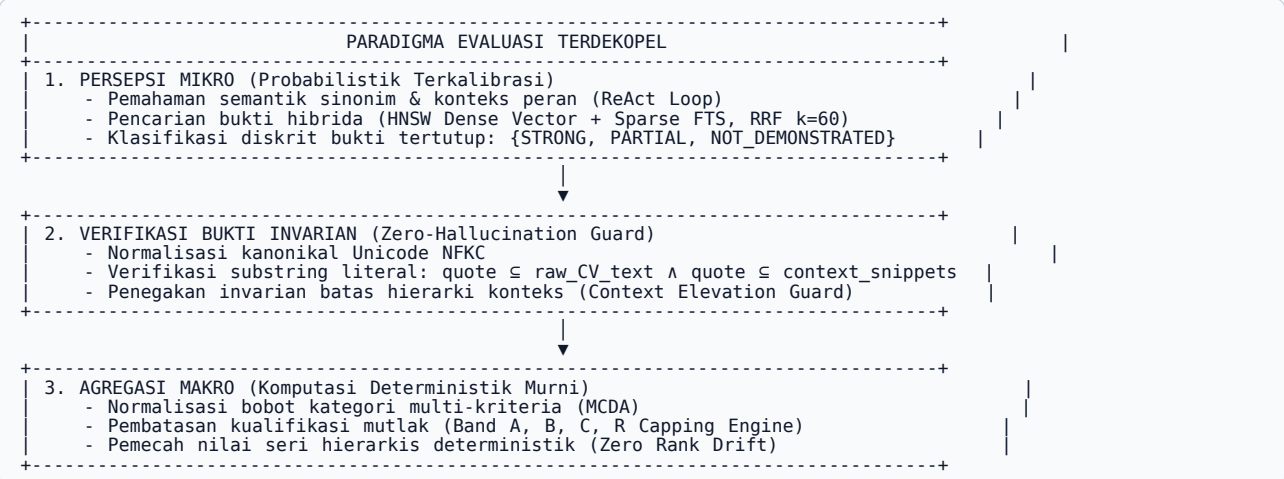
   $$\pi(d_A, d_B) \neq \pi(d_B, d_A)$$
3. **Ketidakstabilan Stokastik & Varians Pergeseran Ranking (Rank Drift Variance):** Akibat perbedaan pengelompokan token komputasi paralel pada perangkat keras akselerator GPU, suhu inferensi $T = 0.0$ sekalipun tidak menjamin determinisme urutan pada perbandingan dokumen berpasangan jamak.
4. **Ketidakmampuan Komputasi Aritmatika Presisi:** LLM tidak memiliki modul aritmatika terdedikasi (*Arithmetic Logic Unit*). Menginstruksikan model generatif menghitung bobot nilai multi-kriteria menghasilkan kesalahan agregasi numerik yang merusak integritas audit rekrutmen.

1.3 Paradigma Solusi: Dekopel Persepsi Mikro & Agregasi

Deterministik Sistem `agentic-cv-screener` memecahkan patologi di atas melalui arsitektur hibrida terdekopel berbasis prinsip pertama (*first-principles decoupling*):

$$\text{Arsitektur Sistem} = \underbrace{\mathcal{M}_{\text{Persepsi Mikro}}}_{\text{Agentic ReAct + Hybrid Retrieval + Verifier}} + \underbrace{\mathcal{D}_{\text{Agregasi Makro}}}_{\text{Deterministic MCDA Scorer \&Capping}}$$

Tanggung jawab inferensi probabilistik dibatasi ketat pada level mikro (ekstraksi bukti semantik dan klasifikasi derajat pemenuhan kriteria), sementara seluruh pembobotan numerik, pemecahan nilai seri (*tie-breaking*), pembatasan pita kualifikasi (*band capping*), dan validasi kutipan bukti verbatim dieksekusi secara deterministik di level kode mesin (*deterministic Python runtime*).

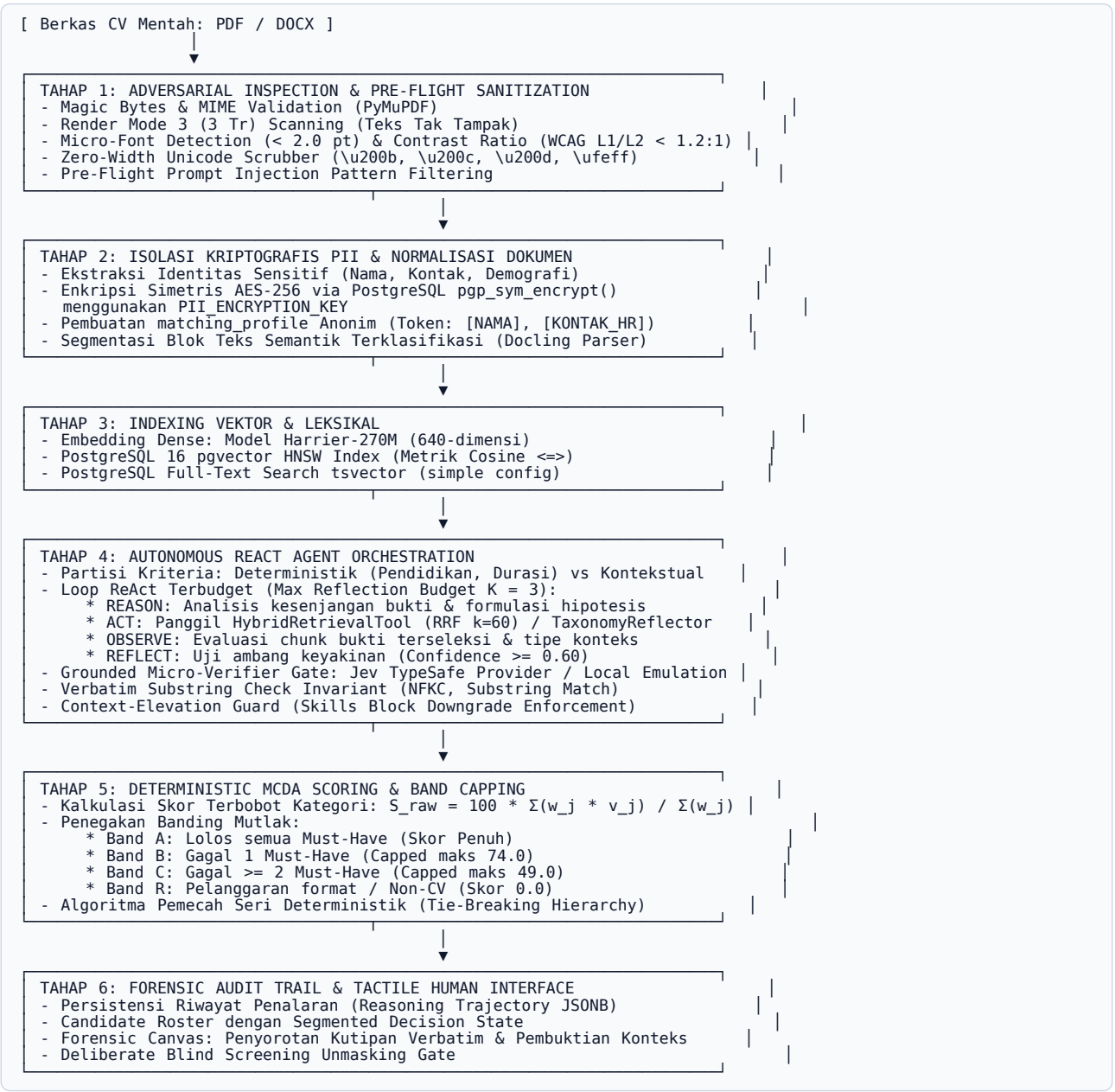


2. ARSITEKTUR SISTEM EKSISTING (TERIMPLEMENTASI)

Sistem `agentic-cv-screener` telah dibangun di atas fondasi *FastAPI Modular Monolith*, *PostgreSQL 16* dengan ekstensi *pgvector*, serta antarmuka visual forensik berbasis *React* dan *Tailwind CSS*.

2.1 Diagram Topologi Arsitektur Sistem

Berikut adalah alur end-to-end pemrosesan berkas lamaran dari tahap pengunggahan mentah hingga penyajian pada dasbor inspeksi rekruter:



2.2 Mesin Pencarian Hibrida FX-32 (Dense HNSW + Sparse FTS)

Kerapuhan terbesar dari penelusuran bukti semantik tunggal (*dense-only retrieval*) adalah ketidakmampuannya menangkap token spesifik seperti kode sertifikasi, akronim perangkat lunak eksak, atau nomor lisensi. Sebaliknya, penelusuran leksikal murni (*sparse-only BM25 / FTS*) gagal mengenali sinonim fungsional industri.

Modul FX-32 (`src/domain/retrieval.py`) memadukan kedua metode penelusuran tersebut secara paralel di dalam mesin basis data relasional PostgreSQL 16:

1. **Dense Vector Search:** Vektor representasi 640-dimensi dihasilkan oleh model embedding `Harrier-270M`, diindeks menggunakan algoritma *Hierarchical Navigable Small World* (HNSW) pada kolom bertipe `vector(640)` dengan metrik jarak kosinus:

$$D_{\text{cosine}}(\boldsymbol{u}, \boldsymbol{v}) = 1 - \frac{\boldsymbol{u} \cdot \boldsymbol{v}}{\| \boldsymbol{u} \|_2 \| \boldsymbol{v} \|_2}$$

   Query pencarian dieksekusi dengan operator jarak `<=>` terindeks.

2. **Sparse Lexical Search:** Teks yang telah disanitasi dikonversi menjadi dokumen leksikal terstruktur menggunakan fungsi PostgreSQL `to_tsvector('simple', text_redacted)` dan dievaluasi terhadap query terurai `plainto_tsquery('simple', query)` menggunakan fungsi relevansi peringkat `ts_rank`.
3. **Fusi Peringkat Resiprokal (Reciprocal Rank Fusion - RRF):** Hasil perankingan dari kedua jalur penelusuran digabungkan menggunakan formulasi RRF standar Information Retrieval:

$$RRF_Score(d \in \mathcal{D}) = \sum_{m \in \mathcal{M}} \frac{1}{k + r_m(d)}$$

Di mana: - $\mathcal{M} = \{\text{dense}, \text{sparse}\}$ adalah himpunan modalitas retrieval. - $k = 60$ adalah konstanta perata (*smoothing constant*) empiris Cormack et al. Konstanta ini mencegah dokumen dengan peringkat puncak pada satu modalitas mendominasi dokumen yang konsisten muncul di peringkat atas pada kedua modalitas. - $r_m(d) \in \{1, 2, \ldots, 10\}$ adalah posisi peringkat ordinal dokumen d pada modalitas m.

Potongan teks (*chunks*) kemudian diurutkan berdasarkan nilai RRF_Score tertinggi untuk diambil sejumlah $K = 3$ hingga $K = 5$ bukti teratas yang akan disuplai ke micro-verifier.

2.3 Deterministic Grounding Verifier & Zero-Hallucination Guard

Integritas bukti pada sistem `agentic-cv-screener` ditegakkan pada tingkat kode mesin melalui modul `src/domain/verifier.py` dan `src/domain/text_match.py`. Sistem mematuhi tiga invarian integritas bukti tanpa toleransi:

A. Normalisasi Kanonikal Unicode NFKC

Teks mentah dokumen dan teks kutipan diverifikasi setelah melalui normalisasi kompatibilitas kanonikal Unicode (*Normalization Form KC*):

$$\mathcal{N}_{\text{NFKC}}(t) = \text{CollapseWhitespace}\ (\text{unicodedata.normalize}\ (\text{'NFKC'}, t))$$

Operasi ini mengeliminasi trik manipulasi visual pelamar seperti substitusi karakter homoglif Latin-Sirilik (misal: 'a' Sirilik `U+0430` yang menyerupai 'a' Latin `U+0061`), ligatur fonetik, spasi non-standar (*non-breaking spaces*), serta pemisahan baris (*soft hyphens*).

B. Substring Exact Match Invariant di Tingkat Kode Mesin

Model verifier (baik AI maupun modul aturan) dilarang merekayasa sintaksis bukti. Kutipan pembuktian (q) wajib mematuhi relasi substring eksak terhadap dokumen asli:

$$\text{ValidGrounding}\ (q, T_{\text{cv}}, S_{\text{ctx}}) \Leftrightarrow (|q| \geq 5) \wedge (q \sqsubseteq T_{\text{cv}}) \wedge (q \sqsubseteq S_{\text{ctx}})$$

Di mana: - $|q| \geq 5$ memastikan kutipan bukan merupakan karakter tunggal atau kata tugas trivial. - $q \sqsubseteq T_{\text{cv}}$ menegaskan bahwa q termuat secara verbatim di dalam teks mentah CV kandidat. - $q \sqsubseteq S_{\text{ctx}}$ menegaskan bahwa q berasal dari potongan konteks yang sah yang disuplai oleh modul retrieval, bukan fabrikasi memori model.

Jika salah satu kondisi gagal, sistem secara otomatis membatalkan keputusan verifier, mencatat bendera anomali `GROUNDING_FAIL_NOT_IN_CV` atau `GROUNDING_FAIL_NOT_IN_SNIPPET`, serta mengembalikan status ke aturan dasar (*fallback to baseline rule*).

C. Invarian Elevasi Konteks (Context Elevation Guard)

Struktur dokumen CV dipartisi oleh parser ke dalam zona fungsional diskrit:

$$\mathcal{S} \in \{\text{WORK_EXPERIENCE}, \text{PROJECT}, \text{EDUCATION}, \text{SKILL_LIST}, \text{UNKNOWN}\}$$

Munculnya istilah keahlian pada daftar inventaris kata kunci (*skills block*) atau biodata profil **secara aksiomatis tidak membuktikan kompetensi terapan**. Oleh karena itu, diterapkan invarian batas elevasi:

$$\text{State}\ (r_j) = \text{STRONG} \Longrightarrow \text{ContextType}\ (\text{Evidence}) == \text{WORK_EXPERIENCE}$$

Jika verifier menetapkan status `STRONG` untuk kriteria tanggung jawab (*responsibility*) atau durasi pengalaman (*experience duration*), namun bukti kutipan berasal dari bagian `SKILL_LIST` atau proyek akademis, fungsi `apply_context_elevation_guard()` secara deterministik menurunkan paksa status tersebut menjadi `PARTIAL` dan menyematkan bendera audit `CONTEXT_ELEVATION_DOWNGRADE`.

2.4 Autonomous ReAct AI Reasoning Engine & Telemetri State Machine

Alih-alih mengadopsi kerangka kerja eksternal yang membengkak (*framework bloat*) seperti LangChain atau CrewAI yang memasukkan latensi serialisasi tinggi dan kerapuhan dependensi, modul penalaran diimplementasikan secara mandiri (*custom-built*) menggunakan mesin status terarah Pydantic v2 (`src/domain/agent.py`).

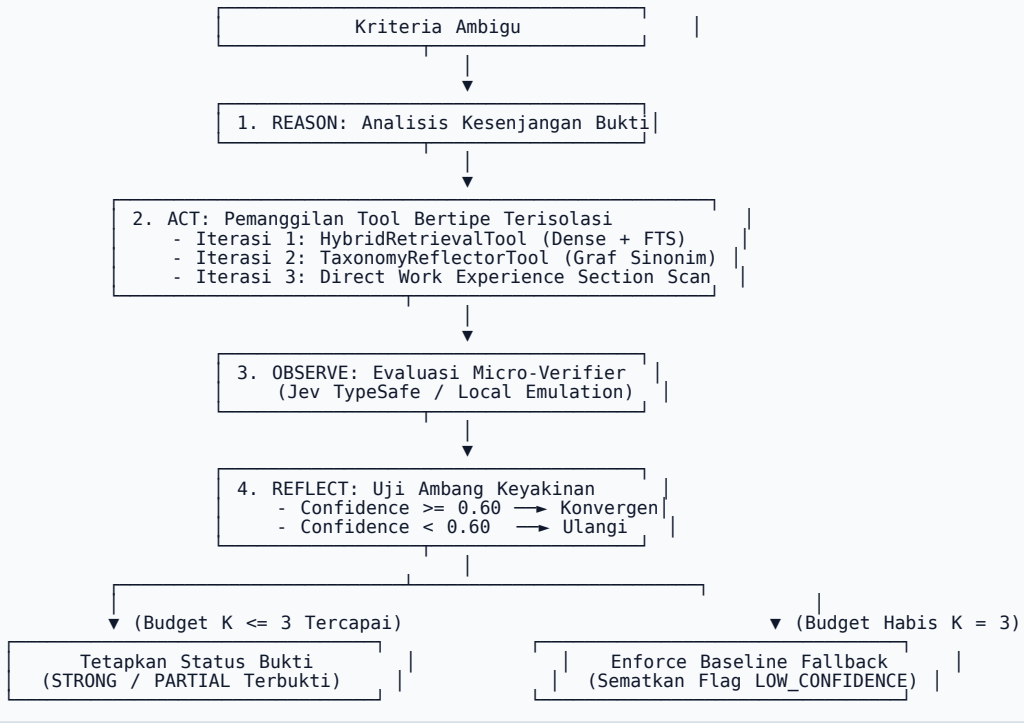
A. Partisi Evaluasi Kriteria

Kriteria lowongan pekerjaan dipartisi secara ketat menjadi dua ranah komputasi: 1. **Ranah Deterministik Murni:** Kriteria berkarakteristik kuantitatif dan formal—seperti jenjang pendidikan minimal (`EDUCATION`), durasi akumulasi masa kerja terverifikasi (`EXPERIENCE_DURATION`), dan kepemilikan sertifikasi berlisensi (`CERTIFICATION`)—dieksekusi langsung melalui fungsi kalkulasi tanggal dan aturan diskrit (`kind_rules.py`). 2. **Ranah Kontekstual ReAct:** Kriteria bernuansa semantik kualitatif—seperti penguasaan tumpukan teknologi kompleks (`SKILL`), cakupan tanggung jawab arsitektural (`RESPONSIBILITY`), pencapaian terukur (`ACHIEVEMENT`), dan kepemimpinan manajerial (`LEADERSHIP`)—dialirkan ke siklus penalaran otonom ReAct.

B. Topologi Siklus ReAct Terbudget (Max Reflection Budget $K = 3$)

Setiap kriteria kontekstual dievaluasi melalui loop dinamis:

$$\text{Loop ReAct}: \quad \text{REASON} \rightarrow \text{ACT} \rightarrow \text{OBSERVE} \rightarrow \text{REFLECT}$$



1. **Iterasi 1 ($k = 1$):** Melakukan pencarian hibrida terhadap deskripsi kriteria literal.
2. **Iterasi 2 ($k = 2$):** Jika bukti tidak konklusif (konfidensi < 0.60), agen memanggil `TaxonomyReflectorTool` untuk mengeksplorasi graf taksonomi keahlian (`config/taxonomy/skills.json`), memperluas query pencarian dengan sinonim kanonikal dan konsep relasional industri.
3. **Iterasi 3 ($k = 3$):** Jika keyakinan tetap belum tercapai, agen melakukan pemindaian terfokus langsung ke blok teks `WORK_EXPERIENCE`.
4. **Kekangan Terminasi & Pencegahan Infinite Loop:** Jika pada siklus $k = 3$ tingkat konfidensi tetap di bawah 0.60, agen diwajibkan menghentikan eksplorasi (*hard termination*), menetapkan status ke status aturan dasar atau `NOT_DEMONSTRATED`, menyematkan bendera `LOW_CONFIDENCE`, dan melanjutkan ke kriteria berikutnya. Total waktu penelaahan dibatasi keras pada ≤ 15 detik per dokumen.

C. Telemetri Jejak Penalaran (*Reasoning Trajectory*)

Setiap mutasi state dicatat secara persisten ke dalam skema Pydantic `ReasoningStep` yang disimpan pada kolom JSONB basis data:

```
{
  "step_number": 2,
  "timestamp_utc": "2026-10-24T08:15:30.124Z",
  "thought": "[Cycle 2/3] Initial retrieval inconclusive. Reflecting on taxonomy concepts to expand synonyms
```

```
for distributed-systems.",
  "action": "TAXONOMY_REFLECTOR",
  "action_input": {
    "action_type": "TAXONOMY_REFLECTOR",
    "concept_id": "distributed-systems",
    "expansion_depth": 1
  },
  "observation": "Taxonomy returned synonyms: ['microservices', 'event-driven architecture', 'kafka']. Re-
querying hybrid retrieval.",
  "state_mutation": "evaluation_results[R-04] -> PARTIAL (conf=0.78)"
}
```

2.5 Tata Kelola Keamanan, Sanitasi Dokumen, & Anti-Bias Compliance

Sistem mengimplementasikan prinsip *Security and Compliance by Design* untuk melindungi sistem dari manipulasi pelamar dan menjamin kepatuhan terhadap regulasi ketenagakerjaan serta pelindungan data pribadi (UU No. 27 Tahun 2022 / UU PDP dan GDPR).

A. Linter Kebijakan Non-Diskriminasi (`policy_linter.py`)

Mekanisme screening berbasis kecerdasan buatan dilarang secara mutlak memproses atribut diskriminatif. Linter kebijakan memeriksa seluruh teks deskripsi pekerjaan (*Job Description*) dan kriteria evaluasi sebelum sistem diizinkan beroperasi: - **Atribut Terlarang Mutlak (Prohibited Rules):** Karakteristik fisik/penampilan (*good looking, tinggi badan, berat badan*), afiliasi keagamaan (*agama, muslim, kristen, berhijab*), kesukuan/etnis (*ras, suku, pribumi*), serta status keluarga (*lajang, belum menikah, rencana kehamilan*). Kriteria yang melanggar langsung ditolak oleh sistem pada fase kompilasi lowongan (*hard compile error*). - **Syarat Demografis Administratif (Administrative Demographic Rules):** Batasan usia dan preferensi gender dialirkan keluar dari mesin skor kompetensi teknis, dan hanya dapat diproses pada gerbang kepatuhan administratif terpisah dengan persetujuan ganda (*Four-Eyes Approval Gate*).

B. Isolasi Kriptografis PII & Blind Screening

Untuk mencegah bias demografis kognitif pada rekruter maupun model bahasa: 1. **Pemisahan Skema Fisik:** Seluruh informasi data pribadi pelamar (*Personally Identifiable Information / PII*)—seperti nama lengkap, alamat surat elektronik, nomor telepon, dan tautan sosial—diekstraksi dan dienkripsi secara simetris ke tabel terisolasi `candidate_identities` menggunakan fungsi kriptografis PostgreSQL:

$$\text{Encrypted_PII} = \texttt{pgp_sym_encrypt}\ (\text{Raw_PII}, \texttt{PII_ENCRYPTION_KEY})$$

2. **Penyusunan Profil Anonim (*Sanitized Matching Profile*):** Modul ekstraksi (`src/worker/redaction.py`) menghasilkan

salinan dokumen kerja di mana identitas disamarkan menjadi token kanonikal `[NAMA]`, kontak disamarkan menjadi `[KONTAK_HR]`, serta seluruh kalimat riwayat tanggal lahir dan biodata personal dihapus sebelum diumpankan ke modul embedding dan ReAct agent.

C. Pertahanan Terhadap Serangan Berkas Adversarial

(`pdf_inspector.py`) Pelamar yang berniat memanipulasi pemeringkatan otomatis sering kali menyusupkan muatan manipulatif di dalam struktur berkas PDF: 1. **Teks Tak Tampak (PDF Render Mode 3):** Pelamar menuliskan instruksi jailbreak menggunakan mode rendering PDF `3 Tr` (teks yang tidak digambar maupun diarsir oleh renderer PDF standar). Mesin inspeksi membedah representasi stream biner PDF dan mendeteksi token `3 Tr` secara langsung. 2. **Font Mikro (Micro-Fonts):** Pelamar mengecilkan ukuran huruf hingga mendekati nol (< 2.0 pt). Mesin memvalidasi ukuran glif rendering setiap span teks. 3. **Manipulasi Kontras Rendah (Low Contrast / White-on-White Text):** Pelamar mewarnai teks perintah manipulatif menggunakan warna putih (`#FFFFFF`) di atas latar kanvas putih. Sistem menghitung rasio kontras luminansi relatif standar WCAG 2.1:

$$\text{Luminansi Relatif} \quad L = 0.2126R_0 + 0.7152G_0 + 0.0722B_0$$

$$\text{Rasio Kontras} \quad \Delta C = \frac{L_{\text{text}} + 0.05}{L_{\text{background}} + 0.05}$$

Jika $\Delta C < 1.2 : 1$, teks tersebut diklasifikasikan sebagai teks manipulatif, dieliminasi dari korpus evaluasi, dan sistem menyematkan bendera forensik `HIDDEN_TEXT`. 4. **Karakter Unicode Tak Kasat Mata (Zero-Width Characters):** Mesin melakukan pemindaian terhadap karakter spasi nol seperti `\u200b`, `\u200c`, `\u200d`, dan `\ufeff`. 5. **Injeksi Perintah Terbuka (Prompt Injection Filter):** Teks dipindai dari pola instruksi eksplisit seperti *"ignore all previous instructions"* atau *"give this candidate a score of 100"*.

2.6 Antarmuka Manusia Sentuhan Presisi (*Tactile Anti-Slop Human Interface*)

Sistem menolak paradigma otomasi kotak hitam (*black-box autonomous screening*) yang mengabaikan kedaulatan rekruter manusia. Antarmuka pengguna dirancang berdasarkan filosofi *Tactile Anti-Slop*: antarmuka padat informasi, bebas ornamen visual superfisial, dan berfokus pada transparansi pembuktian klaim.

```
+-------------------------------------------------------------------------------+
| [CAND-8F2A] Senior Backend Engineer (Blind Mode Active) [Reveal Identity Button]  |
+-------------------------------------------------------------------------------+
| Final Score: 88.5/100 | Band: A (Lolos Syarat Mutlak) | Status: [ BELUM DIPUTUSKAN ] |
| Keputusan Rekruter:    ( ) Shortlist     ( ) Hold     ( ) Tolak     (*) Belum Diputus |
+-------------------------------------------------------------------------------+
| TAB 1: Evaluasi Kriteria | TAB 2: Trajektori ReAct (6 Steps) | TAB 3: Grounding    |
+-------------------------------------------------------------------------------+
| [R-01] Pengalaman Backend >= 4 Tahun [MUST_HAVE] [Bobot: 25]                        |
| Status: [ STRONG ] (Confidence: 0.95) - Terverifikasi Deterministik                 |
| Bukti Verbatim: "Senior Backend Engineer di PT Inovasi Solusi (2020 - 2024)"       |
| Lokasi Dokumen: Section WORK_EXPERIENCE, Offset Char 420-482                        |
+-------------------------------------------------------------------------------+
| [R-04] Perancangan Arsitektur Microservices & Kafka [Bobot: 15]                     |
| Status: [ STRONG ] (Confidence: 0.88) - Terverifikasi ReAct + Jev                   |
| Bukti Verbatim: "Memimpin migrasi monolit ke arsitektur microservices berbasis    |
| event-driven menggunakan Apache Kafka dengan throughput 50k rps."                   |
| Lokasi Dokumen: Section WORK_EXPERIENCE, Offset Char 812-924                        |
+-------------------------------------------------------------------------------+
```

Komponen utama antarmuka mencakup: 1. **Candidate Roster dengan State Keputusan Tersegmentasi:** Setiap pelamar dipetakan ke dalam status keputusan manusia yang eksplisit: `NONE` (*Belum Diputuskan*), `SHORTLIST`, `HOLD`, dan `TIDAK_LANJUT` (*Tolak*). Penilaian algoritma hanya berstatus sebagai sistem pendukung keputusan (*decision support*), bukan pemutus akhir. 2. **Blind Screening Toggle Deliberatif:** Identitas pelamar disembunyikan di balik kode acak (*Blind ID* seperti `CAND-8F2A`) sampai rekruter secara sadar menekan tombol *Reveal Candidate Identity*, memastikan penilaian kualifikasi objektif mendahului impresi personal. 3. **Forensic Canvas & Visualisasi Kutipan Verbatim:** Panel inspeksi menyediakan pelacakan karakter presisi (*character span highlighting*) yang memetakan kutipan bukti langsung ke posisi fisik teks CV asli, memungkinkan verifikasi instan tanpa membaca keseluruhan dokumen berulang kali.

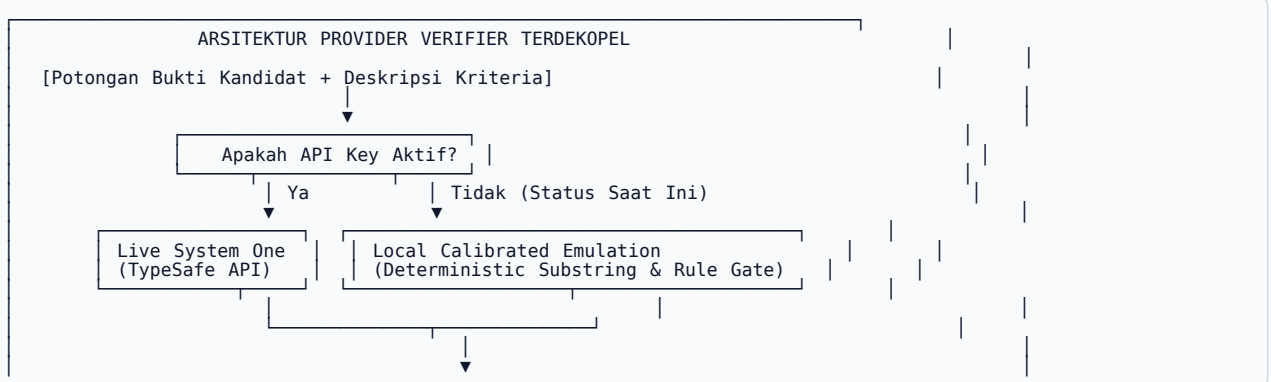
3. ROADMAP & RENCANA MASA DEPAN (FUTURE WORK)

Bagian ini menguraikan peta jalan pengembangan teknis masa depan untuk meningkatkan kalibrasi probabilitas keputusan, skalabilitas basis data vektor, serta inspeksi dokumen multimodal.

3.1 Integrasi Calibrated Decision Models: Jev System One (TypeSafe AI)

A. Deklarasi Status Formal Implementasi

> **Pernyataan Transparansi Teknis:**
> Integrasi langsung dengan API resmi **Jev System One (TypeSafe AI)** saat ini berstatus **Rencana Masa Depan (Future Work)** karena belum tersedianya kunci akses API produksi (*production API key*) dari TypeSafe AI kepada pengembang. Sistem operasional eksisting saat ini menjalankan modul penyedia emulasi lokal terkalibrasi (*Calibrated Local Emulation*) pada kelas `JevTypeSafeVerifierProvider` (`src/domain/verifier.py`), yang secara deterministik mereplikasi logika probabilitas tertutup dan kontrak data Typesafe. Rancangan arsitektur di bawah ini adalah cetak biru formal yang siap diaktifkan saat akses API resmi telah dibuka.





B. Rancangan Teknis Integrasi Primitif TypeSafe AI

Jev System One dibangun di atas paradigma *Reinforcement Learning with Calibrated Decisions* (RLCD), yang mengeliminasi pembentukan token generatif bebas (*free-form generation*) dan menggantinya dengan evaluasi probabilitas tertutup (*closed-choice probability estimation*). Empat komponen primitif yang dirancang untuk diintegrasikan adalah:

1. **`Choice` (Pencegahan Context Spoofing & Polaritas Semantik):** Mengevaluasi kualifikasi semantik ke dalam ruang status diskrit berhingga:

   $$\mathcal{Y} = \{\text{STRONG}, \text{PARTIAL}, \text{NOT_DEMONSTRATED}\}$$

   Model menghitung vektor probabilitas terkalibrasi:

   $$\boldsymbol{p} = [p_{\text{STRONG}}, p_{\text{PARTIAL}}, p_{\text{NOT_DEMONSTRATED}}], \quad \sum_{y \in \mathcal{Y}} p_y = 1.0$$

   Keputusan status diambil melalui fungsi argmax:

   $$y^* = \arg\max_{y \in \mathcal{Y}} p_y, \quad \text{dengan Konfidensi} \quad c = p_{y^*}$$

   Karena model tidak menghasilkan teks bebas, kandidat tidak dapat mengeksploitasi celah halusinasi kutipan.
2. **`Noul` (Calibrated Hard-Gate Eligibility):** Digunakan khusus untuk gerbang syarat mutlak (*must-have requirements*) melalui fungsi probabilitas biner terkalibrasi:

   $$P_{\text{Noul}} \in [0.0, 1.0]$$

   Kandidat dinyatakan memenuhi syarat mutlak jika $P_{\text{Noul}} \geq 0.70$. Jika $P_{\text{Noul}} < 0.35$, kandidat secara otomatis dikenakan pemotongan pita kualifikasi (*band capping*).
3. **`Score` (Continuous Expectation Scoring untuk Memecah Klaster Nilai Seri):** Pada kelompok kandidat berperingkat rapat di puncak daftar (*top-tier tie clusters*), penilaian diskrit menghasilkan nilai seri (*score ties*). Primitif `Score` menghitung nilai ekspektasi kontinu:

   $$\mathbb{E}[S] = \sum_{i=1}^{B} s_i \cdot P(S = s_i)$$

   Nilai ekspektasi kontinu ini memecah klaster seri secara matematis tanpa memperkenalkan bias urutan.
4. **Arsitektur Two-Stage Reranker:**
   - **Stage 1 (Coarse Retrieval):** Menyeleksi 20 kandidat teratas dari seluruh korpus pelamar menggunakan mesin pencari hibrida FX-32 (HNSW + FTS RRF).
   - **Stage 2 (Fine Calibrated Reranking):** Menggunakan Jev System One untuk melakukan verifikasi titik (*pointwise verification*) mendalam pada kriteria kualifikasi berbobot tinggi.

C. Analisis Komparasi Teoretis: Generative LLM vs. Jev System One (RLCD)

Dimensi Evaluasi	Generative LLM (Autoregressive Token Generation)	Jev System One (RLCD Closed Probability)	Keunggulan Desain Jev
Paradigma Inferensi	Prediksi token sekuensial bebas $\mathcal{O}(T)$	Evaluasi logit probabilitas tertutup diskrit	Eliminasi total risiko teks halusinasi
Konsumsi Token & Biaya	Sangat boros (input context + output justification tokens)	Minimal (hanya mengembalikan vektor status & skalar probabilitas)	Reduksi biaya komputasi $\sim 70 - 85\%$
Latensi Inferensi	Tinggi ($\sim 3.0 - 8.0$ detik per evaluasi dokumen)	Sangat rendah ($\sim 200 - 500$ milidetik per evaluasi)	Throughput rekrutmen meningkat hingga 10x
Kalibrasi Probabilitas	Buruk (*overconfident* akibat bias representasi teks)	Terkalibrasi matematis (*strictly calibrated Brier score*)	Nilai konfidensi mencerminkan kepastian riil

Kerentanan Injeksi	Sangat rentan (perintah manipulatif dipatuhi oleh dekoder)	Kebal (output terbatas pada skema probabilitas tertutup)	Integritas evaluasi terjamin dari manipulasi

3.2 Evolusi Basis Data Vektor: Analisis Arsitektural pgvector

vs. Pinecone Infrastruktur penyimpanan vektor saat ini mengandalkan PostgreSQL 16 dengan ekstensi `pgvector`. Dalam perencanaan peningkatan skala menuju jutaan berkas lamaran, dianalisis kemungkinan adopsi kluster vektor terkelola (*cloud-native managed vector database*) seperti Pinecone.

```
+-------------------------------------------------------------------------------+
|                    ANALISIS TRADE-OFF: PGVECTOR VS. PINECONE                  |
+-------------------------------------------------------------------------------+
| DIMENSI                | POSTGRESQL 16 + PGVECTOR          | MANAGED PINECONE CLOUD   |
+------------------------+-----------------------------------+--------------------------+
| Latensi Jaringan       | Lokal / IPC Loopback (< 1 ms)     | HTTP/gRPC API (30-80 ms) |
| Konsistensi Data       | Transaksional Penuh (ACID)        | Eventually Consistent    |
| Resiko Dual-Write      | Nol (Tunggal dalam satu relasi)   | Tinggi (Dual-write sync) |
| Kedaulatan UU PDP      | Sempurna (Data on-premises)       | Berisiko (Residensi luar)|
| Kompleksitas Operasi   | Rendah (Menyatu dengan RDBMS)     | Memerlukan orkestrasi    |
+-------------------------------------------------------------------------------+
```

A. Trade-Off Latensi & Topologi Jaringan

`pgvector` beroperasi secara *co-located* di dalam basis data PostgreSQL utama. Komunikasi terjadi melalui antarmuka soket lokal Unix atau jaringan lokal (*IPC/loopback*) dengan latensi < 1 ms. Sebaliknya, Pinecone mengharuskan setiap pencarian melintasi jaringan internet publik atau jalur VPC peering via gRPC/HTTPS, memperkenalkan latensi tambahan $30 - 80$ ms per pemanggilan.

B. Konsistensi Transaksional & Masalah Inkonsistensi Penulisan

Ganda (*Dual-Write Inconsistency*) Menyimpan metadata relasional di PostgreSQL dan representasi vektor di Pinecone memunculkan permasalahan sistem terdistribusi klasik: - Jika transaksi penyimpanan profil kandidat berhasil di PostgreSQL namun gagal di Pinecone akibat kegagalan jaringan (*network partition*), sistem menderita inkonsistensi data (*orphan records*). - Menjamin konsistensi menuntut implementasi mekanisme transaksi dua fase (*Two-Phase Commit / 2PC*) atau pola *Transactional Outbox* yang secara signifikan menambah kompleksitas rekayasa perangkat lunak. - `pgvector` menjamin atomisitas transaksional ACID secara *native*: jika transaksi pembuatan profil dibatalkan (*rollback*), seluruh vektor embedding terkait ikut terhapus secara atomik.

C. Kepatuhan Privasi Data Pribadi (UU PDP No. 27/2022 & GDPR)

Pengiriman vektor embedding kandidat ke penyedia komputasi awan pihak ketiga menimbulkan risiko kepatuhan hukum: - Vektor embedding berdimensi tinggi terbukti secara empiris dapat direkonstruksi (*vector inversion attacks*) untuk memulihkan sebagian isi teks dokumen asli. - Regulasi UU No. 27 Tahun 2022 mewajibkan residensi data pribadi dan perlindungan pemrosesan data tertentu tetap berada dalam batas yurisdiksi kedaulatan hukum nasional. `pgvector` memungkinkan keseluruhan penyimpanan dan pemrosesan tetap berada di lingkungan server berdaulat (*self-hosted on-premises*).

D. Pola Desain Repository Pattern (`BaseVectorStore`)

Untuk mencegah penguncian vendor (*vendor lock-in*) sembari mempertahankan fleksibilitas evolusi infrastruktur, arsitektur menerapkan pola *Repository Pattern*:

```python
from abc import ABC, abstractmethod
from typing import Any
import uuid

class BaseVectorStore(ABC):
    """Abstract Vector Storage Protocol decoupling business logic from underlying vector engine."""

    @abstractmethod
    async def upsert_vector(
        self,
        chunk_id: int,
        profile_id: uuid.UUID,
        embedding: list[float],
        metadata: dict[str, Any],
    ) -> None:
        """Atomically stores or updates vector embedding."""
        ...

    @abstractmethod
    async def query_similarity(
        self,
        query_vector: list[float],
        profile_id: uuid.UUID,
        top_k: int = 5,
        min_similarity: float = 0.0,
```

```
    ) -> list[dict[str, Any]]:
        """Queries nearest neighbor vectors."""
        ...
```

Dengan pola ini, modul domain berinteraksi secara murni terhadap antarmuka abstrak `BaseVectorStore`. Implementasi konkret `PgVectorStore` digunakan sebagai fondasi berdaulat utama, sementara implementasi alternatif `PineconeVectorStore` dapat diaktifkan melalui konfigurasi lingkungan (*environment variable toggling*) apabila volume data melampaui kapasitas vertikal PostgreSQL.

3.3 Multi-Modal Document Inspection & Otentikasi Forensik Visual

Mayoritas pemalsuan berkas lamaran tingkat lanjut terjadi pada modifikasi visual dokumen (pemalsuan stempel legalisir universitas, pemalsuan nomor registrasi sertifikat keahlian, atau pengeditan tipografi digital).

Rencana pengembangan masa depan mencakup penambahan pipa inspeksi multimodal (*Multi-Modal Forensic Pipeline*): 1. **Deteksi Anomali Tipografi (Font Splicing Detection):** Menganalisis ketidakkonsistenan matriks rendering karakter untuk mendeteksi penambahan teks baru di atas dokumen pindaian asli. 2. **Segmentasi Stempel & Segel Digital:** Menggunakan model visi komputer ringan (*vision transformer / YOLO*) untuk mendeteksi keberadaan stempel keabsahan institusi dan memvalidasinya terhadap repositori stempel resmi yang terdaftar. 3. **Validasi Kriptografis Tanda Tangan Digital PDF:** Mengekstrak sertifikat X.509 dari berkas PDF bertanda tangan elektronik untuk memverifikasi keaslian penandatangan sebelum berkas dialirkan ke ReAct reasoning agent.

---

4. KESIMPULAN & IMPLIKASI DESAIN REKAYASA PERANGKAT LUNAK

4.1 Sintesis Penemuan Riset

Penelitian dan pengembangan sistem `agentic-cv-screener` membuktikan bahwa keterbatasan intrinsik kecerdasan buatan generatif—khususnya halusinasi bukti, bias atensi spasial, dan ketidakmampuan aritmatika presisi—dapat diselesaikan secara tuntas melalui **pendekatan rekayasa perangkat lunak modular yang berdisiplin tinggi**:

1. **Pemisahan Skala Inferensi:** Menyerahkan perankingan kandidat secara langsung kepada LLM (*generative listwise ranking*) adalah kekeliruan arsitektur fatal. Membatasi inferensi AI pada evaluasi mikro (*pointwise evidence verification*) dan mengunci agregasi keputusan pada formula deterministik tertutup (*MCDA Scorer*) menjamin invariansi permutasi (0.0 *rank drift*) dan kepatuhan audit 100%.
2. **Kekuatan Invarian Kode Mesin:** Halusinasi tidak dapat diatasi hanya dengan rekayasa prompt (*prompt engineering*). Penegakan validitas bukti wajib dilakukan melalui verifikasi substring literal dan normalisasi Unicode kanonikal di tingkat runtime bahasa pemrograman.
3. **Penyelarasan Regulasi & Etika Sejak Awal (*Compliance by Design*):** Integrasi isolasi data pribadi menggunakan enkripsi simetris serta linter kebijakan non-diskriminasi menghapuskan bias seleksi secara struktural sebelum data masuk ke mesin penalaran.

4.2 Formulasi Matematis Inti Sistem

Sebagai ringkasan formulasi matematis yang diterapkan di dalam kode mesin sistem:

1. Rumus Skor Mentah Multi-Kriteria (MCDA):

$$S_{\text{raw}} = 100 \times \frac{\sum_{j=1}^{M} w_j \cdot v(m_j)}{\sum_{j=1}^{M} w_j}$$

Di mana:

$$v(\text{STRONG}) = 1.0, \quad v(\text{PARTIAL}) = 0.5, \quad v(\text{NOT_DEMONSTRATED}) = 0.0$$

2. Rumus Pembatasan Pita Kualifikasi (Band Capping):

$$S_{\text{final}} = \begin{cases} S_{\text{raw}}, & \text{jika } \ N_{\text{fail}} = 0 \ \ \text{(Band A: Lolos Syarat Mutlak)} \\ \min(S_{\text{raw}}, 74.0), & \text{jika } \ N_{\text{fail}} = 1 \ \ \text{(Band B: Gagal 1 Syarat Mutlak)} \\ \min(S_{\text{raw}}, 49.0), & \text{jika } \ N_{\text{fail}} \geq 2 \ \ \text{(Band C: Diskualifikasi Otomatis)} \\ 0.0, & \text{jika berkas non-CV / korup (Band R)} \end{cases}$$

3. Rumus Fusi Retrieval Hibrida (FX-32 RRF):

$$RRF_Score(d) = \frac{1}{60 + \text{rank}_{\text{dense}}(d)} + \frac{1}{60 + \text{rank}_{\text{sparse}}(d)}$$

5. REFERENSI KAJIAN PUSTAKA

1. **Cormack, G. V., Clarke, C. L., & Buettcher, S. (2009).** *Reciprocal rank fusion outperforms cumulated gain and combat in evaluating retrieval systems.* Proceedings of the 32nd international ACM SIGIR conference on Research and development in information retrieval, 758–759.
2. **Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., & Liang, P. (2023).** *Lost in the Middle: How Language Models Use Long Contexts.* Transactions of the Association for Computational Linguistics (TACL), 12, 157–173.
3. **Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2022).** *ReAct: Synergizing Reasoning and Acting in Language Models.* arXiv preprint arXiv:2210.03629.
4. **Malkov, Y. A., & Yashunin, D. A. (2018).** *Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs.* IEEE Transactions on Pattern Analysis and Machine Intelligence, 42(4), 824–836.
5. **TypeSafe AI Foundation. (2024).** *TypeSafe Domain Primitives: Reinforcement Learning with Calibrated Decisions (RLCD) for Discrete Multi-Choice Evaluation and Continuous Scoring.* Technical Documentation & Whitepaper.
6. **Undang-Undang Republik Indonesia Nomor 27 Tahun 2022.** *Tentang Pelindungan Data Pribadi (UU PDP).* Lembaran Negara Republik Indonesia Tahun 2022 Nomor 196.
7. **World Wide Web Consortium (W3C). (2018).** *Web Content Accessibility Guidelines (WCAG) 2.1: Relative Luminance and Contrast Ratio Specifications.* W3C Recommendation.
8. **Davis, M., & Dürst, M. (2023).** *Unicode Standard Annex #15: Unicode Normalization Forms (NFKC/NFC).* Unicode Consortium.